



Afina tu propia IA

Pablo García Pérez
David Martín Gascuña

¿Qué haremos hoy?



Entender

Qué significa afinar un modelo y cuándo compensa frente a prompting o RAG.



Preparar

Un modelo base abierto y tus datos propios para hacer supervised fine-tuning con pares {entrada → salida correcta}



Afinar

Veremos el método PEFT para hacer fine-tuning de una red. Nos enfocaremos en la familia LoRa (*Low Rank Adaptation*).



Competir

Reto Marcha/Pare: aplica lo aprendido.

Entender

¿Afinar un modelo?

Entender · Cómo especializar un LLM



Prompting

Interacción con el modelo sin tener que tocar su arquitectura interna. Aquí encontramos los few-shot (ejemplos exactos de entrada y salida) o el chain of thought (razonar paso a paso)



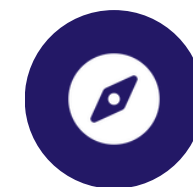
RAG

Recuperar documentos y añadirlos al prompt. Conocimiento fresco sin tocar los pesos.



Fine-tuning

Reentrenar al modelo modificando los pesos de sus neuronas utilizando nuevos datasets para especializarlo en una tarea. Existe el *full fine-tuning* o el *parcial fine-tuning*.



¿Cuándo afinar?

Cuando necesitas un comportamiento estable que el prompt por sí solo no logra, es cuando empezamos a considerar el ajuste fino.

Entender · El truco: Low Rank Adaptation (LoRA)



Congelar la base

No reentrenamos los miles de millones de parámetros del modelo original. Sería muy costoso y una pérdida de tiempo. Por ello, decimos que “congelamos” parte de la red, que significa no tocar esos pesos.



Adaptadores LoRA

En lugar de tocar la red al completo, añadimos unas redes neuronales extras (adaptadores) pegadas a las capas del modelo original. Durante el entrenamiento sólo actualizamos los pesos de estos adaptadores.



QLoRA (4-bit)

Quantatized Low Rank Adaptation.

En vez de cargar el modelo completo lo cuantizamos, reduciendo la precisión matemática de los parámetros de 16 bits a 4 bits



Resultado

estas herramientas nos abren un nuevo mundo. Esto significa que el ajuste fino ya no es exclusivo de las Big Tech.

Preparar

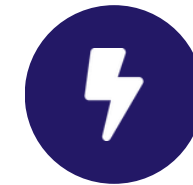
Lo necesario

Preparar · Fine-tuning en abierto



Modelos base

Qwen2.5, Gemma, Phi: pesos en abierto y SLMs. Más documentación en Hugging Face.



Unsloth

Herramienta que nos permite optimizar los cálculos matemáticos que ocurren en la tarjeta gráfica durante el entrenamiento. Esto logra que el proceso sea hasta el doble de rápido.



Hugging Face

Modelos, datasets y las librerías necesarias (Transformers, PEFT, etc.).



Dónde entrenar

Podemos entrenar con nuestro propio cómputo a través del uso de nuestras GPUs o bien usar la nube en caso de falta de cómputo. Exploraremos la opción local, donde tenemos control de cómo usamos nuestros recursos.

Preparar · Tus datos mandan



Formato instrucción: pares {instrucción → respuesta}.



Calidad > cantidad: 100 ejemplos buenos rinden más que 10 000 ejemplos ruidosos.



Plantilla de chat: respetamos el formato conversacional típico que espera el modelo base que usamos.

Afinar



Asociación de
Inteligencia Artificial

¡A entrenar!

Afinar · El pipeline con LoRA



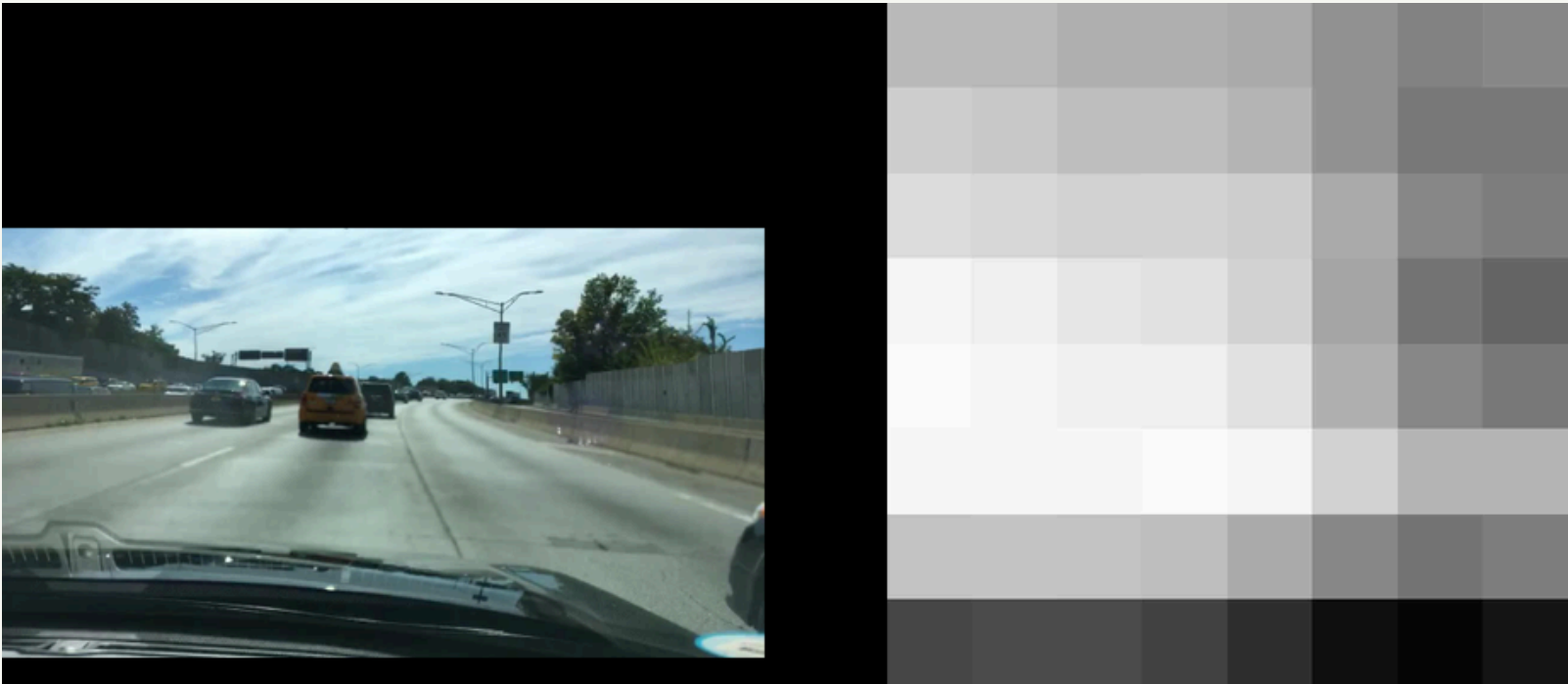
Jugaremos con los materiales después de que se explique el reto Marcha/Pare

<https://github.com/maybepueblo/6JulioseLIA.git>

Competir

Reto Marcha/Pare

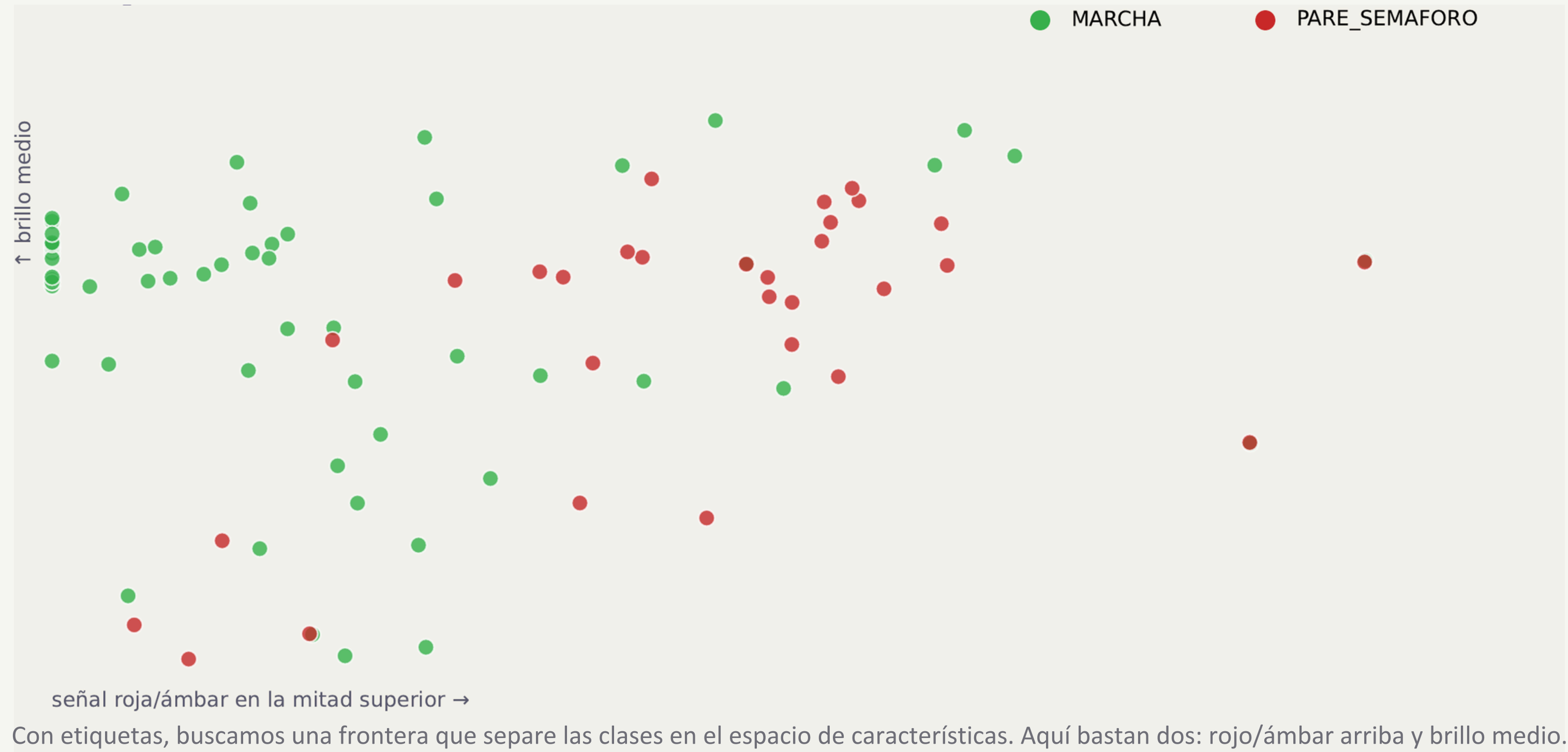
La imagen · son números



Cada píxel es un número (0–255). El modelo no ve una escena: ve una matriz. Ese es el punto de partida de toda la visión por computador.

189	185	178	177	172	147	131	139
208	202	193	191	182	149	123	123
221	216	210	213	207	172	137	129
246	240	230	227	213	166	119	104
250	247	242	240	226	180	136	123
248	248	249	250	246	212	183	183
196	199	196	191	174	138	119	128
71	76	75	67	48	16	7	22

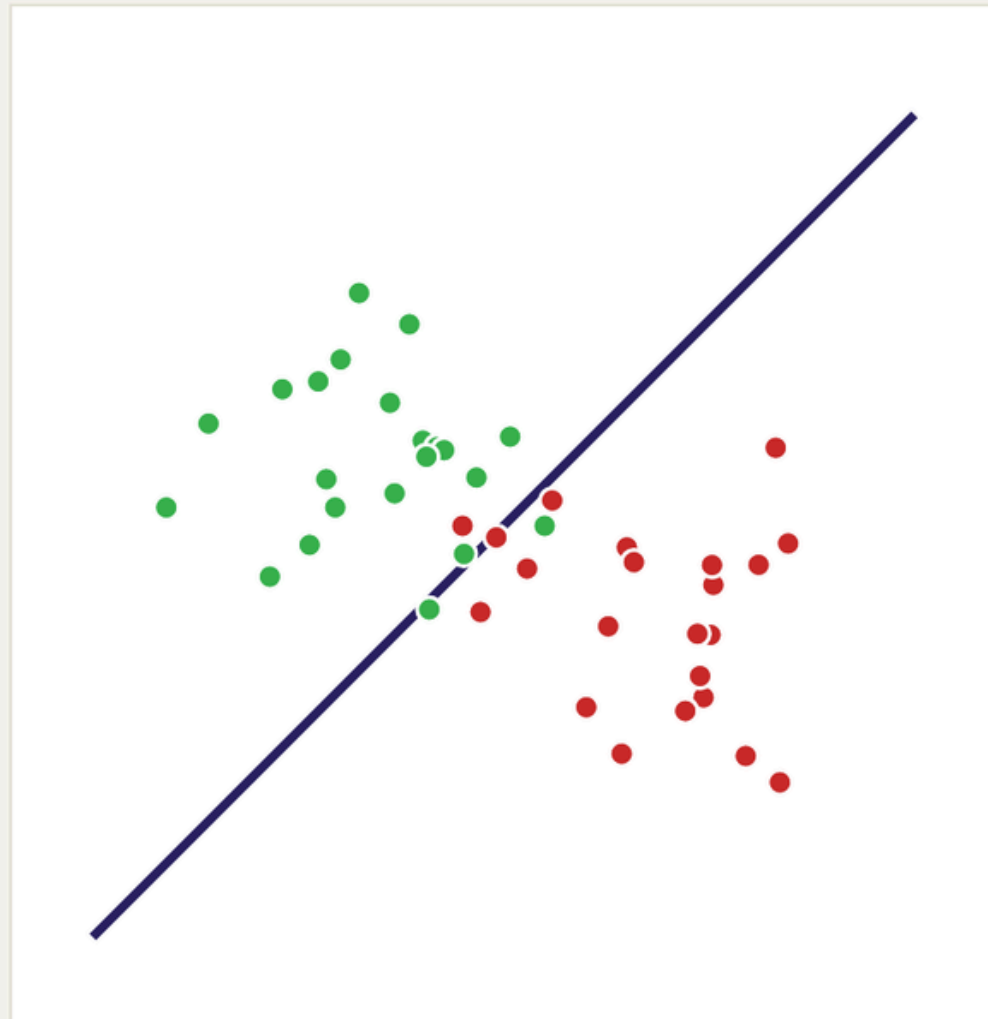
Supervisado · dos características, una frontera



Compromiso en predicción: sesgo frente a varianza (regresión)

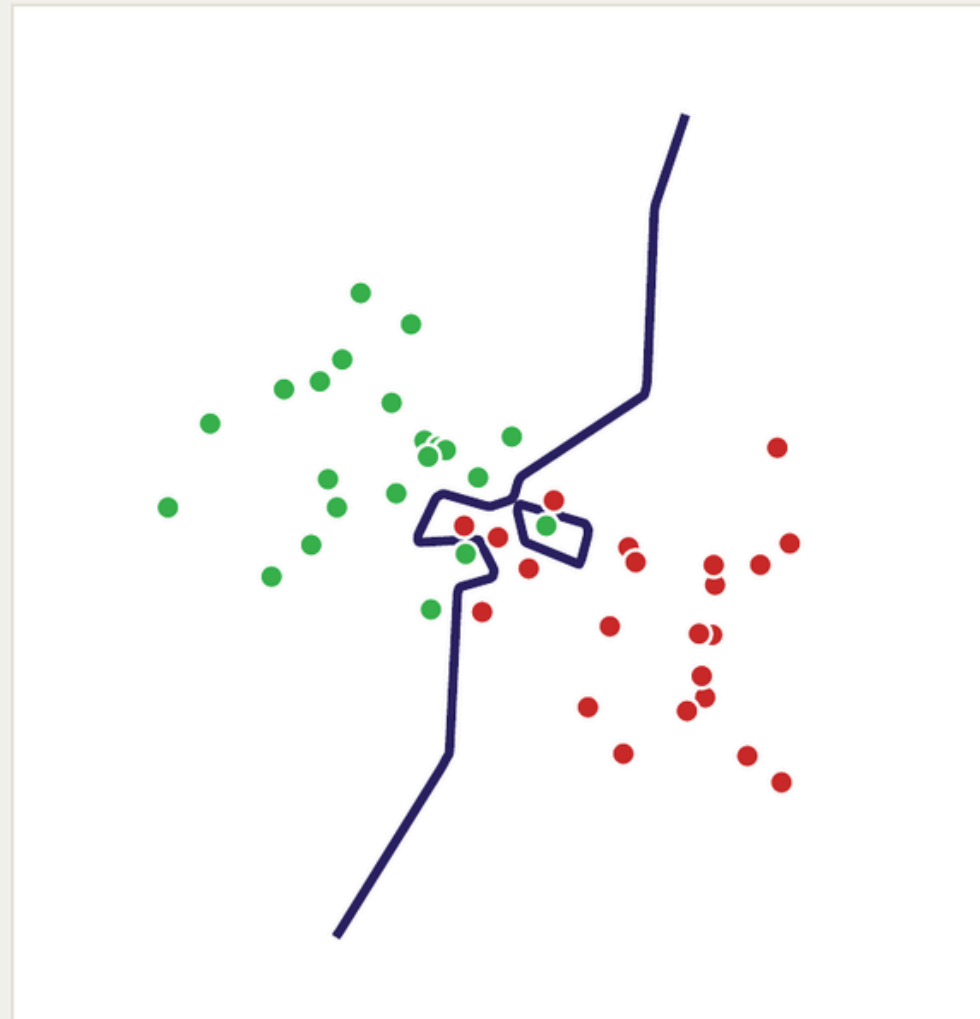


Compromiso en predicción: sesgo frente a varianza (clasificación)



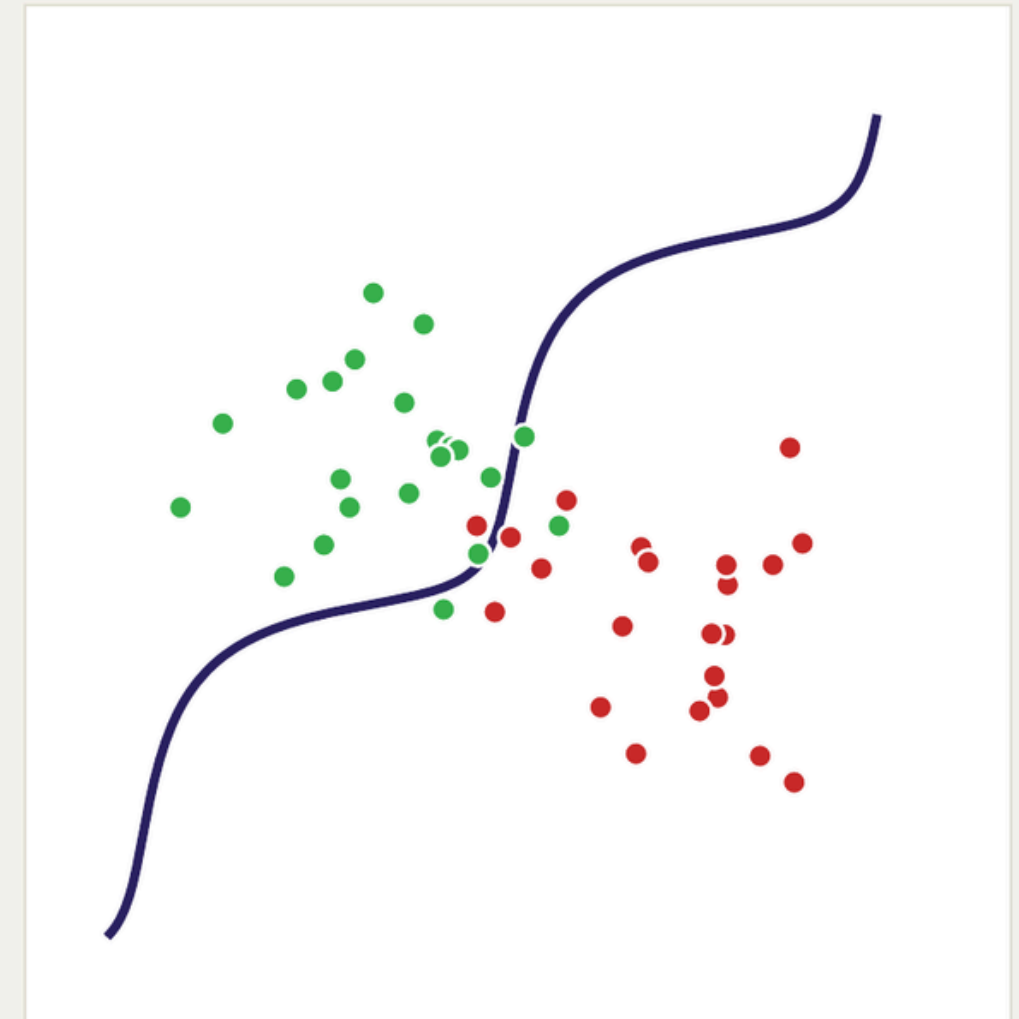
recta

misma frontera para todo el espacio



memoriza

una isla por cada ejemplo raro



compromiso

sigue la forma general, no cada punto

Redes · el problema de ver toda la imagen a la vez

$$64 \times 64 \times 3 = 12\,288$$

números por imagen

Una sola capa densa de 128 neuronas ya son **1 572 992 parámetros**.

Y cada neurona ve todos los píxeles a la vez: no distingue «cerca» de «lejos» dentro de la imagen.

Redes de kernels · el mismo filtro, en toda la imagen



El mismo filtro (los mismos pesos, en naranja) se aplica en cada posición.

Localidad

Cada salida mira solo un trocito de la imagen, no todos los píxeles.

Pesos compartidos

El mismo filtro (aquí, un borde vertical) sirve en cualquier zona.

Competir · El reto Marcha/Pare



MARCHA



PARE_SEÑAL



PARE_SEMÁFORO



PARE_PEATÓN



PARE_VEHÍCULO

La tarea Clasificar cada imagen de conducción: seguir en MARCHA o el motivo para PARAR.

Métrica F1-macro: las cinco clases pesan igual; ignorar una rara sale caro.

5 clases MARCHA · PARE_SEÑAL · PARE_SEMÁFORO · PARE_PEATÓN · PARE_VEHÍCULO.



Entrada

$x \in [0,1]^{(64 \times 64 \times 3)}$

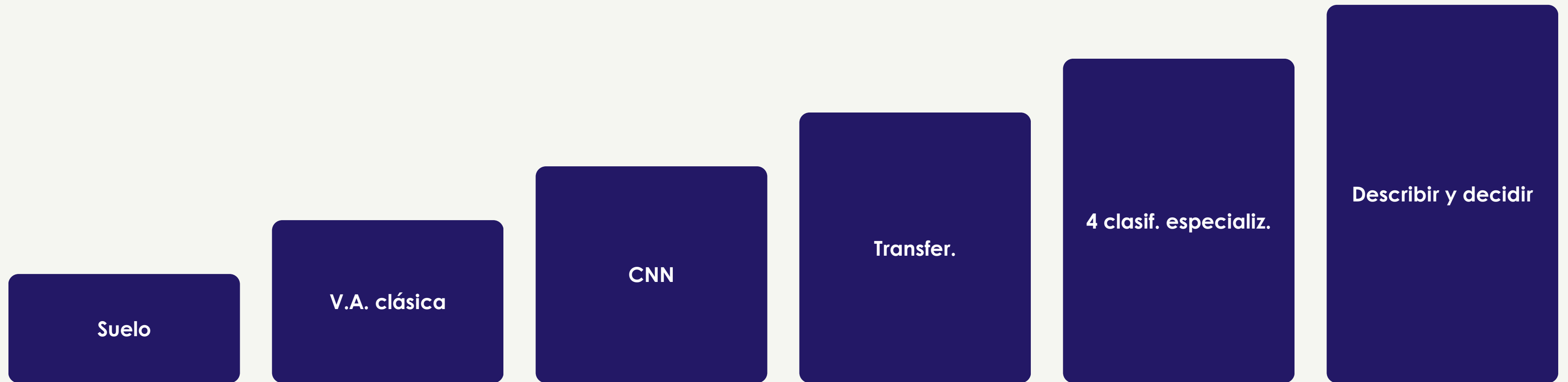
imagen reescalada, 3 canales



Salida

5 clases: MARCHA o el motivo de parada (señal, semáforo, peatón, vehículo)

El reto · la escalera de complejidad



Ocho preguntas al mismo problema, todas con el mismo test. No hay un ganador único: hay cuánta calidad compra cada milisegundo.

Arquitecturas · red plana frente a convolución



Red plana (FCNN)

Aplana la imagen en un vector. Sin noción de «cerca de»: permutar los píxeles no le cambia nada.



Convolución (CNN)

Localidad + pesos compartidos. Menos parámetros y sensible a dónde está cada cosa.



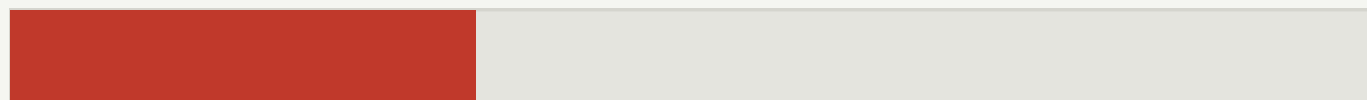
En números

Con una arquitectura adaptada (convolución) procesar sale más barato que la fuerza bruta de una red plana igual de sensible.

Heurística · el rojo estándar de señal y semáforo



PARE_SEMÁFORO



0,34% de la imagen es rojo saturado



PARE_SEÑAL



0,57% de la imagen es rojo saturado

poco rojo → semáforo (círculo pequeño) · mucho rojo → señal (octógono grande)

Clasificadores independientes · cada uno responde una pregunta



¿mucho rojo?

señal

→ **PARE**



¿parecido a algo visto?

vecino más cercano

→ **MARCHA**



¿qué ve un detector?

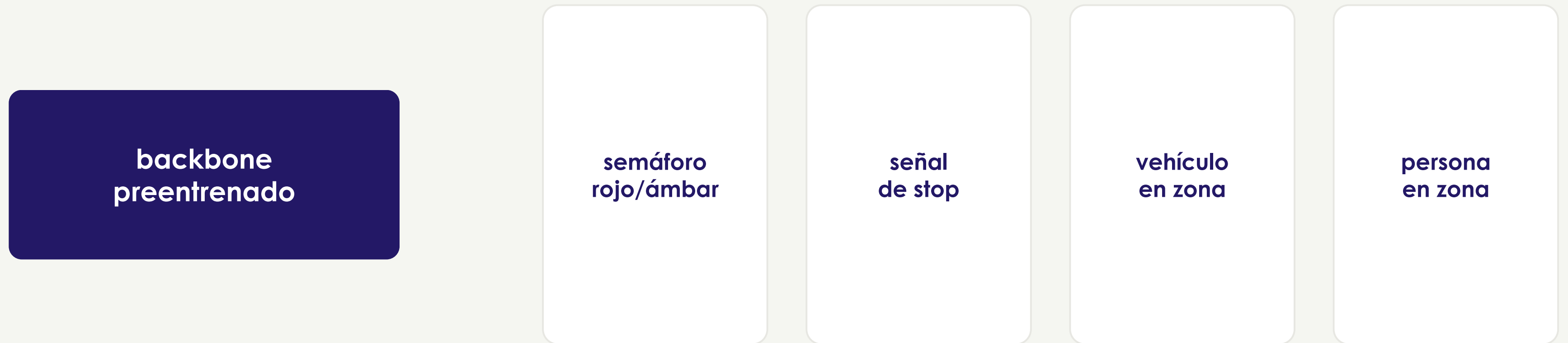
detector + reglas de zona

→ **PARE**

PARE (2 votos a 1)

Solo suma si las preguntas son independientes entre sí.

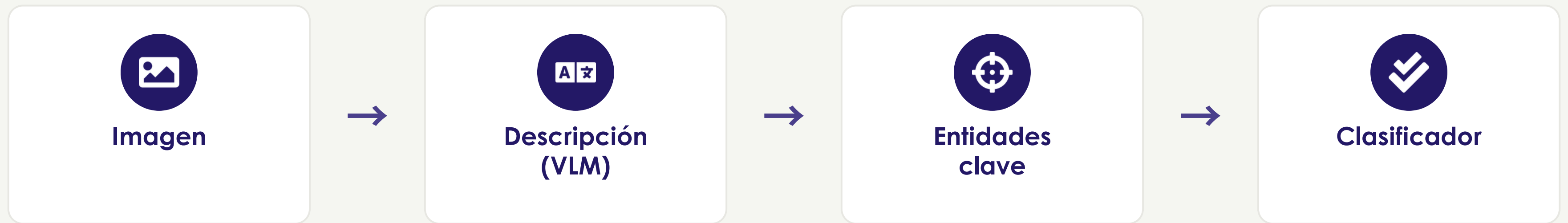
Sugerencia · cuatro cabezas, una red preentrenada



Cada cabeza se afina por separado, con sus propios ejemplos.

Nuestro código (S5): un único detector genérico (COCO) + reglas de zona, sin afinar nada.

Sugerencia · describir la imagen para clasificarla



El lenguaje como cuello de botella: de ~1 MB de píxeles a una frase corta.
La decisión se toma sobre las entidades mencionadas, no sobre los píxeles.

Es el mismo modelo de lenguaje que afináis en el taller de LLM, aplicado aquí a describir.

El reto · cómo medimos: F1-macro y recall



F1-macro (métrica primaria)

$$F1 = 2 \cdot (P \cdot R) / (P + R)$$

media armónica de precisión (P) y recall (R): solo es alta si ambas lo son.

$$F1\text{-macro} = (1/5) \cdot \sum_k F1_k$$

promedia las 5 clases sin ponderar por frecuencia: fallar en una clase rara hunde la media igual que fallar en una común.



recall_PARE (seguridad)

$$\text{recall} = TP / (TP + FN)$$

de todos los PARE reales, ¿cuántos detectamos?

Un falso negativo (FN) = **crear MARCHA cuando había que PARAR**. Es el error peligroso: no frenar. Por eso vigilamos el recall sobre PARE por encima de la precisión.



Coste en empate gana la solución más eficiente: nº de parámetros, tamaño del modelo y latencia por imagen.



¡Gracias!

Ahora les toca a ustedes: afinen sus propios modelos.